Fast Image Search over ImageNet-1k: What Approximate Indexing and Quantization Buy

CAP6938 Homework 2

Longho Bernard Che
University of Central Florida
USA

Abstract

This project builds a search system that returns the 10 images most similar to a query image from the 1.28 million ImageNet-1k training images. It then measures what approximate indexing and quantization gain and lose compared with exact search. An IVF index is 105× faster than exact search with the same precision. Adding product quantization makes search 241× faster and the index 37× smaller, for a loss of 0.02 in precision.

1 Introduction

Each image is represented by an embedding vector, and the search returns the 10 database vectors closest to the query's vector. *Exact search* (Flat) compares the query with all 1.28M vectors. It is accurate but slow, and it needs about 4 GB of memory. This report tests two standard remedies:

- **Approximate indexing (IVF)** groups the vectors into 4,096 clusters and searches only the *nprobe* clusters nearest to the query. A larger nprobe gives more accurate results but slower search.
- **Quantization** stores compressed vectors. SQ8 keeps 1 byte per dimension (4× smaller). PQ64 splits each 768-dimensional vector into 64 parts of 12 dimensions and stores, for each part, the 1-byte id of the nearest of 256 learned centroids (48× smaller).

2 Method

(1) **Split the data.** The *database* is the whole *train* split (1,281,167 images, 732–1,300 per class). The *queries* are one image per class drawn uniformly at random (seed 42) from the *validation* split, which has 50 images per class, giving 1,000 queries. The *test* split was not used because its labels are not public. Since database and queries come from different splits, a query never finds itself.

(2) **Embed** every image with OpenAI CLIP ViT-L/14 (768 dimensions) and normalise each vector to unit length, so the inner product equals cosine similarity. Embedding took 0.59 h on a Colab A100 GPU (603 images/s).

(3) **Index** with five FAISS settings that cover exact and approximate search, each with and without quantization (Table 2). The IVF clusters, SQ8 value ranges and PQ centroids were learned from 200,000 randomly sampled database vectors, never from queries. IVF-PQ uses the same clusters as IVF-Flat, so the two differ only by quantization.

(4) **Measure** on a Colab CPU runtime, one query at a time. *Latency* is the median and 99th percentile (p99) search time per query, excluding the time to embed the query. *Memory* is the size of the saved index. *Precision@10* (P@10) is the fraction of the top 10 results with the query's class. *Recall@10* (R@10) is the fraction of exact search's top 10 that a setting also returns; it isolates the loss caused by the index and quantization.

(5) **Reproduce.** Table 1 lists every setting that affects the results. The three notebooks read each one from an environment variable, with these values as defaults.

Table 1: Configuration used for all results.

Dataset	`benjamin-paine/imagenet-1k-256x256` (Hugging Face)
Queries	1 per class from *validation*, random seed 42
Model	`open_clip` `ViT-L-14-quickgelu`, `openai` weights
Preprocessing	bicubic resize to 224 px, centre crop, CLIP mean/std; fp16, batch 256
Vectors	768-d, L2-normalised, stored as fp16; inner-product metric
IVF	4,096 lists; nprobe $\in \{1, 2, 4, \ldots, 128\}$, reported at 16
Quantization	SQ8 (8 bits/dim); PQ 64 sub-vectors × 8 bits
Index training	200,000 database vectors, random seed 42
Search	top 10; 50 untimed warm-up queries; 1 query per call
Failure rule	P@10 bad ≤ 0.3, good ≥ 0.7, drop ≥ 0.3
Software	Python 3.13, PyTorch 2.11, `open_clip` 2.32.0, `faiss-cpu` 1.11.0
Hardware	embedding: A100-SXM4-40GB; search: Colab CPU, 2 vCPUs, 2 FAISS threads

Table 2: Results on 1,000 queries, with IVF at nprobe = 16. Speedup is relative to Flat's median latency.

Setting	MB	Med. ms	p99 ms	Speedup	P@10	R@10
Flat (exact)	3,936	366.4	454.1	1×	**.701**	1.000
SQ8 (exact)	984	216.0	472.1	1.7×	**.701**	.981
PQ64 (exact)	83	45.8	86.5	8×	.606	.329
IVF-Flat	3,959	3.5	6.0	105×	**.701**	.975
IVF-PQ64	106	1.5	2.1	**241**×	.682	.492

3 Results

Table 2 compares the five settings, and Figure 1 shows how accuracy and latency change with nprobe.

(1) **IVF buys speed, not memory.** At nprobe = 16, IVF-Flat is 105× faster than exact search, keeps 97.5% of its results, and has the same precision. Raising nprobe beyond 8–16 adds little: going from 8 to 128 lifts recall from 0.950 to 0.998 but makes search 7.8× slower.

(2) **SQ8 buys memory, not speed.** It is 4× smaller with almost the same accuracy, but only 1.7× faster, because it still compares the query with every vector.

(3) **PQ buys both, at a cost in recall.** IVF-PQ is the fastest (1.5 ms) and nearly the smallest (106 MB) setting, and it loses only 0.02 precision. Its recall, however, stays near 0.49 whatever the

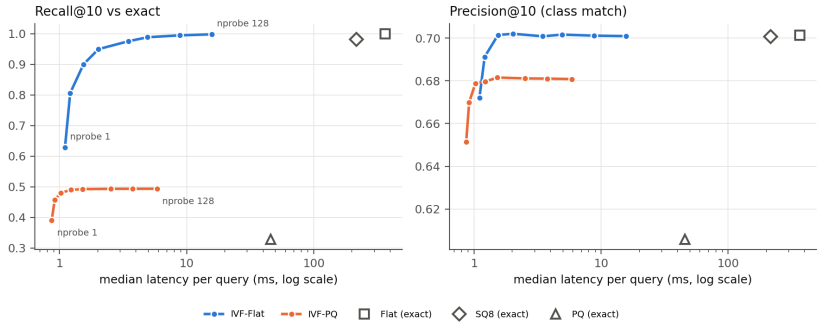


Figure 1: Recall@10 (left) and precision@10 (right) against median latency as nprobe grows from 1 to 128. Exact-search settings are single points.



Figure 2: Failure cases: the query and the top 5 results of the failing setting (✗ = different class). In cases 4–6, exact search returns 10/10 correct results and IVF-PQ 9–10/10.

nprobe. Quantization, not the number of clusters searched, sets that limit.

(4) **IVF-PQ beats plain PQ.** With the same 64-byte codes, IVF-PQ reaches recall 0.49, while PQ alone reaches 0.33. IVF-PQ compresses each vector's offset from its cluster centre, which is smaller than the vector itself and so is encoded more accurately.

(5) **Precision drops much less than recall,** because quantization usually replaces a true neighbour with another image of the same class.

4 Failure Analysis

The cause of each failure was identified by comparing a query's P@10 across settings at nprobe = 16:

- wrong even under exact search (P@10 ≤ 0.3): the *embedding model* or the *label*;
- correct under exact search and IVF-Flat, but P@10 drops by ≥ 0.3 under PQ: *quantization*;
- P@10 drops under IVF-Flat: the *approximate index*. No query fell into this group.

Figure 2 shows six cases:

(1) **Viaduct (label).** A streetcar on a viaduct. The results are streetcars, which really are in the image, but ImageNet allows only one label.

(2) **Eggnog (embedding).** A dim photo of a bottle. The results share only the dark indoor look, not the object.

(3) **American coot (label).** The query is a collage of ducks with no coot in it. The returned ducks are correct.

(4) **Web site (quantization).** PQ kept the house photo on the page and lost the page layout, so it returns houses.

(5) **Thimble (quantization).** PQ placed the thimble among the much more common cup images, so it returns espresso cups.

(6) **Muzzle (quantization).** PQ kept "white shaggy dog" and lost the muzzle, so it returns terriers.

5 Conclusion

Precision is capped at 0.70 by the embedding model, not by the index. For 214 of the 1,000 queries, even exact search finds 3 or fewer correct images. If about 4 GB of RAM is available, IVF-Flat at nprobe 8–16 gives exact-level precision in 2–3.5 ms. If memory is limited, IVF-PQ at nprobe 8–16 needs only 106 MB and about 1.3 ms, for 0.02 lower precision.